1. Qual o Objetivo?

- **Objetivo Geral:** Estudar e implementar diretrizes de acessibilidade em um jogo educacional para auxiliar alunos da área de Computação (incluindo aqueles com deficiência visual) no estudo da linguagem UML, com foco específico em diagramas de casos de uso.
- **Objetivos Específicos:**
    - Estudar conceitos e diretrizes de acessibilidade voltados para a web e jogos interativos.
    - Estudar os conceitos de UML focados em diagramas de casos de uso.
    - Pesquisar ferramentas e mecanismos para o desenvolvimento de jogos sérios e educativos.
    - Desenvolver o protótipo do jogo educacional, interativo e acessível.
    - Realizar a validação dos requisitos de acessibilidade e da experiência do usuário por meio de testes práticos.

2. Metodologia (Como foi desenvolvido o U.C. Battle?)

A metodologia seguiu as seguintes etapas de desenvolvimento e avaliação:

- **Revisão Bibliográfica:** Levantamento teórico cobrindo a modelagem UML (diagramas de casos de uso), jogos sérios/educativos e diretrizes de acessibilidade baseadas na WCAG 2.1 e nos trabalhos de Cheiran & Pimenta (2011).
- **Levantamento de Requisitos:** Definição dos requisitos funcionais do jogo e seleção de 21 diretrizes de acessibilidade a serem implementadas na interface.
- **Desenvolvimento da Aplicação:** Programação do protótipo em ambiente Web utilizando a biblioteca/linguagem React.
- **Testes e Validação:** Aplicação de testes do jogo com 26 estudantes dos cursos de Ciência da Computação e Sistemas de Informação. Após jogarem, os alunos responderam a um formulário com perguntas adaptadas do *Game Experience Questionnaire* (GEQ) e questões focadas na validação dos requisitos de acessibilidade.

3. Qual o Jogo Desenvolvido?

O jogo desenvolvido chama-se **U.C. Battle**.

- **Estilo e Dinâmica:** É um jogo sério educativo em ambiente web onde o jogador assume o papel de um herói com o objetivo de derrotar 5 vilões em sequência para salvar seu mundo.
- **Mecânica de Combate:** O combate acontece por meio da resolução de charadas de múltipla escolha sobre diagramas e conceitos de Casos de Uso em UML. Acertos tiram pontos de vida do vilão, enquanto erros reduzem a vida do herói. O jogador possui uma habilidade especial de dica que pode ser ativada uma vez por batalha se estiver com 2 ou menos pontos de vida.
- **Recursos Adicionais:**
    - **Modo Treino (Tutorial):** Espaço para praticar as charadas sem sofrer penalidades por erros.
    - **Área "Book":** Seção teórica onde os conceitos de casos de uso são apresentados em formato de histórias em quadrinhos.

    - **Menu de Opções:** Permite ativar/desativar músicas, efeitos sonoros e narrações das perguntas, alternativas e histórias.

4. Diretrizes de Acessibilidade, Quais Foram?

Foram estudadas e implementadas 21 diretrizes de acessibilidade baseadas nas recomendações da WCAG 2.1 e de Cheiran & Pimenta (2011), organizadas nos seguintes princípios:

- **Perceptível:**
    - **Narração de conteúdo textual:** Opção de narração em áudio para perguntas, alternativas, dicas e quadrinhos do *book*.
    - **Alternativas textuais:** Apresentação de equivalentes em texto para elementos não textuais.
    - **Contraste de cores:** Cuidados com a relação de contraste entre texto, componentes e fundo da tela.
- **Operável:**
    - **Sem limite de tempo:** O jogador pode responder às questões no seu próprio tempo, sem exigência de respostas rápidas.
    - **Acessibilidade por teclado:** Possibilidade de operar e navegar por todo o jogo utilizando apenas o teclado.
    - **Controle de áudio:** Opção de ligar, desligar ou ajustar individualmente músicas, efeitos e narrações.
- **Compreensível e Previsível:**
    - **Navegação consistente:** Botões e menus repetidos aparecem sempre na mesma ordem e posição em todas as páginas.
    - **Destaque visual de foco:** Indicação clara do elemento em foco, sem mudanças bruscas de contexto.
    - **Modo Treino:** Presença de um tutorial livre de penalidades para aprendizado com os próprios erros.
    - **Títulos claros:** Identificação do objetivo de cada página através de títulos descritivos.
- **Design Inclusivo:**
    - **Uso não exclusivo de cores/formatos:** As informações não são transmitidas apenas por cores ou formas visuais.
    - **Progressão natural de dificuldade:** Aumento gradual do nível de desafio a cada vilão enfrentado.

5. Resultados

- **Validação do Software:** O jogo foi concluído com sucesso e disponibilizado para execução via navegador.
- **Avaliação da Acessibilidade:** Obteve alto grau de aprovação entre os usuários testados:
    - **84,6%** aprovaram a personalização de áudio e narração.
    - **76,9%** elogiaram a ausência de tempo limite para responder.
    - **76,9%** aprovaram a navegação intuitiva e padronizada.
    - **73,0%** aprovaram o aumento gradual de dificuldade.
    - **65,4%** avaliaram positivamente o recurso de recurso de narração de textos.

- **Experiência do Jogador (GEQ):**
    - **84,6%** dos jogadores relataram sentimento de sucesso e **77,0%** sentiram-se desafiados na medida certa.
    - Baixa rejeição e frustração: **80,8%** não se sentiram frustrados e **88,5%** não se irritaram durante a partida.
- **Oportunidades de Melhoria:** Foi observada a necessidade de aprofundar a narrativa e os elementos lúdicos do jogo para aumentar o nível de imersão dos jogadores, além de refinar a paleta de cores para melhorar o contraste visual.

6. Contribuição

- **Inclusão no Ensino de Computação:** Oferece um recurso educacional acessível e gratuito para auxiliar o aprendizado e a prática de diagramas de Casos de Uso (UML), promovendo a inclusão de estudantes com deficiência visual.
- **Guia para Desenvolvedores:** Identifica, aplica e valida na prática um conjunto de critérios de acessibilidade voltados a jogos educativos interativos, servindo de referência para a criação de novos jogos inclusivos.

Atividade Prática: Do Requisito ao Diagrama de Casos de Uso

Disciplina: Engenharia de Requisitos
Tema: UML - Diagrama de Casos de Uso
Sistema Analisado: U.C. Battle

1. Etapa 1: Leitura e Identificação

Perguntas Orientadoras

1. **Qual é o objetivo principal do U.C. Battle?**
   O objetivo principal do U.C. Battle é atuar como um jogo educacional acessível destinado ao ensino de Engenharia de Requisitos, especificamente focado na aprendizagem e fixação dos conceitos de Diagramas de Casos de Uso em UML. No jogo, os estudantes enfrentam vilões através do cumprimento de tarefas pedagógicas e resolução de desafios/charadas.
2. **Quem interage diretamente com o sistema?**
   O ator principal que interage diretamente com o sistema é o **Jogador** (estudante).
3. **Quais funcionalidades são oferecidas ao jogador?**
   As funcionalidades oferecidas incluem: Selecionar fase, Enfrentar vilão, Responder charada, Receber dica, Aceder ao modo de treino, Estudar conceitos, Selecionar tema de estudo, Consultar conteúdo, Configurar recursos de acessibilidade, Render-se e Consultar progresso.
4. **Quais regras condicionam o uso da habilidade de receber uma dica?**
   A habilidade de receber uma dica só pode ser utilizada **uma única vez por batalha** e unicamente quando o jogador possuir **dois pontos de vida (HP) ou menos**.
5. **Quais funcionalidades contribuem para a acessibilidade do jogo?**
   A funcionalidade principal é a de *Configurar recursos de acessibilidade*, que permite adaptar elementos visuais, navegação por teclado/comandos adaptados, ajuste de contraste e suporte a leitores de ecrã para utilizadores com necessidades específicas.

Elemento Identificado	Evidência no Artigo	Classificação
Jogador	"O estudante enfrenta vilões respondendo a perguntas..."	Ator
Selecionar fase	"O software permite selecionar uma fase."	Requisito Funcional
Enfrentar vilão	"...enfrenta vilões respondendo a perguntas..."	Requisito Funcional
Responder charada	"...respondendo a perguntas..."	Requisito Funcional
Regra de uso da Dica	"Somente pode ser utilizada uma vez por batalha e quando o jogador possuir dois pontos de vida ou menos."	Regra de Negócio
Configurar acessibilidade	"Quais funcionalidades contribuem para a acessibilidade do jogo?"	Requisito de Acessibilidade
Consultar progresso	"...permite ao jogador verificar o seu avanço nas etapas."	Requisito Funcional

Quadro de Análise de Requisitos

2. Etapa 2: Construção e Modelação UML

Especificação do Modelo

- **Fronteira do Sistema:** U.C. Battle
- **Ator Principal:** Jogador
- **Casos de Uso Incluídos no Modelo (8):**
  - Selecionar fase

    - Enfrentar vilão
    - Responder charada
    - Receber dica
    - Aceder ao modo de treino
    - Estudar conceitos
    - Configurar recursos de acessibilidade
    - Consultar progresso
- **Relacionamentos Espefícos:**
    - <<include>>: De *Enfrentar vilão* para *Responder charada* (comportamento obrigatório na batalha).
    - <<extend>>: De *Receber dica* para *Responder charada* (comportamento opcional sujeito a condição).
- **Nota UML (Regra de Negócio):** Anexada à relação de extensão de *Receber dica*: "Condição de Extensão: Apenas 1x por batalha e quando Pontos de Vida ≤ 2".

Representação Textual do Diagrama UML (PlantUML)

```
@startuml
left to right direction
skinparam packageStyle rectangle

actor Jogador

rectangle "U.C. Battle" {
  usecase "Selecionar fase" as UC1
  usecase "Enfrentar vilão" as UC2
  usecase "Responder charada" as UC3
  usecase "Receber dica" as UC4
  usecase "Aceder ao modo de treino" as UC5
  usecase "Estudar conceitos" as UC6
  usecase "Configurar recursos de acessibilidade" as UC7
  usecase "Consultar progresso" as UC8

  note right of UC4
    Regra de Negócio:
    Habilidade utilizável apenas 1x por batalha
    e quando o jogador possuir HP <= 2.
  end note
}

Jogador --> UC1
Jogador --> UC2
Jogador --> UC5
Jogador --> UC6
Jogador --> UC7
Jogador --> UC8

UC2 ..> UC3 : <<include>>
```

```
UC4 ..> UC3 : <>
@enduml
```

Discussão das Decisões de Modelação

1. **"Responder charada" faz parte obrigatória de "Enfrentar vilão"?**
   Sim. Durante uma batalha com um vilão, o ato de responder a charadas é indispensável e obrigatório para causar dano ou avançar no confronto. Por isso, utilizou-se a relação <<include>>.
2. **"Receber dica" sempre acontece ou depende de uma condição?**
   Depende estritamente de uma condição. Trata-se de uma funcionalidade opcional acionada pelo utilizador em momentos críticos de dificuldade. Por esse motivo, foi modelada como uma relação <<extend>> sobre o caso de uso *Responder charada*.
3. **"Estudar conceitos" é uma etapa da batalha ou uma funcionalidade independente?**
   É uma funcionalidade independente de consulta pedagógica, acessível a partir do menu do sistema, permitindo ao jogador preparar-se antes de entrar em batalhas.
4. **O modo de treino deve reutilizar algum comportamento da batalha?**
   Sim, o modo de treino reutiliza a resolução de charadas (questões), contudo sem o impacto direto na perda de vida real ou no progresso do mapa de fases do jogo principal.
5. **Quais requisitos de acessibilidade precisam aparecer no modelo?**
   O caso de uso *Configurar recursos de acessibilidade* foi explicitado como interação direta do Jogador para permitir a personalização global da interface e controlos.

3. Etapa 3: Matriz de Rastreabilidade

ID	Requisito do Artigo	Caso de Uso Correspondente	No Diagrama?
RF01	Permitir a seleção da fase de jogo pretendida.	Selecionar fase	Sim
RF02	Permitir o confronto direto com vilões em formato de batalha.	Enfrentar vilão	Sim
RF03	Questões teóricas	Responder charada	Sim

ID	Requisito do Artigo	Caso de Uso Correspondente	No Diagrama?
	durante o jogo.		
RF04	Disponibilizar ajuda/dica quando a vida do jogador for baixa.	Receber dica	Sim
RF05	Permitir adaptação de definições de acessibilidade no jogo.	Configurar recursos de acessibilidade	Sim

4. Checklist de Validação

- **[X]** Os atores representam papéis externos ao sistema (ex: Jogador).
- **[X]** Os casos de uso foram escritos com verbo no infinitivo (ex: Selecionar, Enfrentar, Responder).
- **[X]** A fronteira do sistema está identificada como "U.C. Battle".
- **[X]** Cada associação representa uma interação real.
- **[X]** O <<include>> representa um comportamento obrigatório e reutilizado.
- **[X]** O <<extend>> representa um comportamento opcional ou condicionado.
- **[X]** As regras de negócio foram representadas como notas/condições e não como atores.
- **[X]** Os requisitos selecionados estão devidamente rastreados no diagrama.
- **[X]** A necessidade de acessibilidade foi considerada como caso de uso explícito.
- **[X]** O diagrama está legível, estruturado e sem elementos desconectados.

5. Socialização e Roteiro de Apresentação

Justificativa das Decisões de Modelação (até 5 linhas)

Optou-se por separar as funcionalidades de consulta e configuração das ações de combate para manter a clareza visual. A relação de inclusão entre *Enfrentar vilão* e *Responder charada* garante a obrigatoriedade da componente pedagógica. Por outro lado, a funcionalidade *Receber dica* foi modelada como extensão condicional à resposta de charadas, refletindo com precisão a limitação de vida estipulada na regra de negócio.

Roteiro para Apresentação (Pitch de 2 Minutos)

1. **Apresentação Geral:** O modelo representa o ecossistema do jogo *U.C. Battle* centrado no ator *Jogador*.
2. **Relação <<include>>:** O caso de uso *Enfrentar vilão* inclui obrigatoriamente *Responder charada*, visto que o progresso no combate depende necessariamente de responder a perguntas pedagógicas.
3. **Relação <<extend>>:** O caso de uso *Receber dica* estende *Responder charada* apenas sob a condição de o jogador possuir 2 ou menos pontos de vida, limitada a uma utilização por combate.
4. **Acessibilidade:** Foi incluído o caso de uso *Configurar recursos de acessibilidade* diretamente associado ao Jogador para garantir a adaptação atempada da interface.
5. **Dificuldade Encontrada:** Definir a fronteira exata entre o que é regra de negócio condicional (usada no extend/nota) e o que deveria constituir um caso de uso autónomo.

6. Questão de Encerramento

Que problemas podem surgir durante o desenvolvimento quando um requisito funcional não possui correspondência clara no modelo UML?

Quando um requisito funcional carece de correspondência num modelo UML, surgem vários problemas graves ao longo do ciclo de vida do desenvolvimento de software:

- **Perda de Rastreabilidade e Omissão:** O Requisito pode ser esquecido pela equipa de desenvolvimento e de testes, resultando na entrega de um produto incompleto que não satisfaz as necessidades do cliente.
- **Erros de Arquitetura e Retrabalho:** A ausência do funcionalidade no diagrama dificulta o planeamento de módulos do sistema, levando a refatorações dispendiosas na fase de codificação.
- **Impacto nas Estimativas e Custos:** Dificulta a medição correta do esforço e complexidade do projeto, resultando em desvios de prazos e custos acrescidos.
- **Inconsistência na Testagem:** A equipa de QA não conseguirá desenhar casos de teste adequados por falta de alinhamento entre a documentação de requisitos e a modelação do sistema.